

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

Programación III

TRABAJO PRÁCTICO INTEGRADOR

FOOD STORE

Sistema de Gestión de Pedidos

Alumno:
Christian Emmanuel Olivero

Profesor: Ariel Enferrel

Fecha:
02 de Julio 2026

Contenido

Introducción	4
Objetivos	5
Objetivo General	5
Objetivos Específicos.....	5
3. Marco Teórico	6
3.1 Programación Orientada a Objetos (POO)	6
3.2 Java	6
3.3 Gradle	7
3.4 JPA (Java Persistence API)	8
3.5 Hibernate.....	8
3.6 H2 Database	9
3.7 JPQL (Java Persistence Query Language)	9
3.8 TypeScript.....	10
3.9 Vite	10
3.10 HTML5, CSS3 y LocalStorage	11
4. Arquitectura del Sistema y Decisiones Técnicas	12
4.1 Arquitectura General del Proyecto	12
4.2 Arquitectura del Backend.....	13
4.3 Implementación del BaseRepository	15
4.4 Repositorios Especializados	16
4.5 Relaciones entre Entidades	16
4.6 Persistencia mediante Hibernate.....	17
4.7 Arquitectura del Frontend.....	17
Pages	18
Services.....	18
Types	18
Utils	18
4.8 Uso de LocalStorage	19
4.9 Organización Modular.....	19
4.10 Justificación de las Decisiones Técnicas.....	19
5. Desarrollo del Backend.....	20
5.1 Introducción	20
5.2 Modelo de Dominio.....	21
5.3 Gestión de Categorías	21
5.4 Gestión de Productos	22

5.5 Gestión de Usuarios	22
5.6 Gestión de Pedidos.....	23
5.7 Persistencia de Datos	23
5.8 Consultas JPQL	24
5.9 Validaciones Implementadas	24
5.10 Menú Principal	25
5.11 Resultados Obtenidos	25
6. Desarrollo del Frontend	26
6.1 Introducción	26
6.2 Organización del Proyecto	27
6.3 Sistema de Autenticación.....	27
6.4 Catálogo de Productos	28
6.5 Detalle del Producto.....	29
6.6 Carrito de Compras	29
6.7 Historial de Pedidos.....	30
6.8 Panel de Administración	30
6.9 Gestión de Categorías	31
6.10 Gestión de Productos	31
6.11 Gestión de Pedidos.....	31
6.12 Uso de LocalStorage	32
6.13 Experiencia de Usuario.....	32
6.14 Resultados Obtenidos	33
7. Dificultades Encontradas y Soluciones Implementadas.....	37
7.1 Introducción	37
7.2 Configuración Inicial del Proyecto.....	37
7.3 Modelado de las Relaciones entre Entidades	37
7.4 Implementación del BaseRepository	38
7.5 Consultas JPQL	38
7.6 Persistencia mediante H2.....	39
7.7 Organización del Frontend	39
7.8 Manejo del LocalStorage.....	39
7.10 Validaciones del Sistema.....	40
7.11 Aprendizajes Obtenidos	41
7.12 Reflexión Final	41
8. Conclusiones.....	42
Link Video :.....	43

Link Repositorio GitHub:	43
9. Bibliografía y Webgrafía	44

Introducción

El presente documento describe el desarrollo del Trabajo Práctico Integrador correspondiente a la asignatura **Programación III**, cuyo objetivo fue diseñar e implementar una aplicación Full Stack denominada **Food Store**, destinada a la gestión de pedidos de un negocio gastronómico.

El proyecto fue desarrollado siguiendo las consignas establecidas por la cátedra y aplicando los conocimientos adquiridos durante el cursado en relación con programación orientada a objetos, persistencia de datos, desarrollo web y organización modular del software. Para ello se implementó una solución compuesta por dos aplicaciones independientes: un frontend desarrollado con **TypeScript y Vite**, encargado de la interfaz de usuario, y un backend desarrollado en **Java** utilizando **JPA/Hibernate** para la gestión de la persistencia sobre una base de datos **H2**.

La aplicación contempla dos perfiles de usuario claramente diferenciados. Por un lado, el **administrador**, responsable de gestionar categorías, productos y pedidos mediante un panel administrativo. Por otro lado, el **cliente**, quien puede autenticarse, navegar el catálogo de productos, utilizar un carrito de compras y consultar el historial de sus pedidos.

Durante el desarrollo se aplicaron conceptos fundamentales como la separación de responsabilidades, reutilización de código mediante repositorios genéricos, persistencia orientada a objetos, consultas JPQL, manejo de relaciones entre entidades y organización modular del frontend mediante páginas, servicios e interfaces de TypeScript.

Asimismo, el proyecto permitió integrar conocimientos adquiridos en materias anteriores, combinando técnicas de programación orientada a objetos, bases de datos, persistencia y desarrollo web moderno para construir una aplicación funcional, escalable y de fácil mantenimiento.

Finalmente, este documento tiene como propósito describir las decisiones técnicas adoptadas durante el desarrollo, explicar la arquitectura implementada, documentar las principales dificultades encontradas y presentar las funcionalidades más relevantes del sistema desarrollado.

Objetivos

Objetivo General

Desarrollar una aplicación Full Stack para la gestión de un sistema de pedidos gastronómicos, aplicando los conocimientos adquiridos durante la asignatura Programación III mediante el uso de tecnologías modernas para el desarrollo de aplicaciones web y persistencia de datos.

Objetivos Específicos

- Diseñar una arquitectura separando claramente el frontend del backend.
- Implementar un sistema de autenticación basado en roles de usuario.
- Desarrollar un catálogo de productos con funcionalidades de búsqueda, filtrado y ordenamiento.
- Implementar un carrito de compras con persistencia local.
- Desarrollar un panel administrativo para la gestión de categorías, productos y pedidos.
- Aplicar persistencia de datos utilizando JPA e Hibernate.
- Implementar consultas mediante JPQL para resolver operaciones específicas del sistema.
- Modelar correctamente las relaciones entre entidades utilizando anotaciones JPA.
- Aplicar principios de reutilización mediante la implementación de un repositorio genérico.
- Organizar el código siguiendo una estructura modular que facilite su mantenimiento y escalabilidad.
- Documentar el proceso de desarrollo y las decisiones técnicas adoptadas durante la implementación del proyecto.

3. Marco Teórico

3.1 Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) constituye uno de los paradigmas de programación más utilizados en el desarrollo de software moderno. Su objetivo principal es modelar problemas del mundo real mediante objetos que encapsulan datos y comportamientos relacionados, favoreciendo la reutilización del código, la modularidad y el mantenimiento de las aplicaciones.

Este paradigma se fundamenta en cuatro pilares principales:

- **Abstracción:** permite representar únicamente las características relevantes de un objeto, ocultando los detalles innecesarios para su utilización.
- **Encapsulamiento:** protege los datos internos de un objeto mediante modificadores de acceso y métodos públicos, evitando modificaciones indebidas.
- **Herencia:** posibilita que una clase herede atributos y comportamientos de otra, promoviendo la reutilización de código.
- **Polimorfismo:** permite que diferentes clases respondan de manera distinta ante una misma operación, favoreciendo la flexibilidad del sistema.

Durante el desarrollo del backend de Food Store estos principios fueron aplicados en el diseño del modelo de dominio. Todas las entidades heredan de una clase abstracta denominada **Base**, la cual centraliza los atributos comunes del sistema, como el identificador, el estado de eliminación lógica y la fecha de creación. De esta forma se evita la duplicación de código y se facilita el mantenimiento de las entidades.

Asimismo, cada entidad representa un objeto del dominio del problema, como categorías, productos, usuarios y pedidos, modelando las relaciones existentes entre ellos mediante asociaciones propias de la programación orientada a objetos.

3.2 Java

Java es un lenguaje de programación orientado a objetos desarrollado por Sun Microsystems (actualmente Oracle), ampliamente utilizado para el desarrollo de aplicaciones empresariales debido a su robustez, portabilidad y amplio ecosistema de herramientas.

Una de sus principales características es la independencia de la plataforma, ya que el código compilado se ejecuta sobre la Máquina Virtual de Java (JVM), permitiendo que una misma aplicación funcione en diferentes sistemas operativos sin necesidad de modificaciones.

Entre las ventajas más relevantes de Java se destacan:

- Tipado fuerte y estático.
- Recolección automática de memoria (Garbage Collector).
- Gran cantidad de bibliotecas disponibles.
- Amplio soporte para programación orientada a objetos.
- Alto nivel de seguridad y estabilidad.

En este proyecto Java fue utilizado para desarrollar la totalidad del backend, implementando la lógica de negocio, el modelo de entidades, la persistencia mediante JPA/Hibernate y el menú interactivo por consola solicitado por la consigna.

La utilización de Java permitió implementar un código modular y reutilizable, facilitando la separación de responsabilidades entre entidades, repositorios y clases utilitarias.

3.3 Gradle

Gradle es una herramienta de automatización de construcción (Build Tool) utilizada para gestionar proyectos Java y otros lenguajes. Su función principal consiste en administrar dependencias externas, compilar el código fuente, ejecutar pruebas y generar los artefactos finales del proyecto.

A diferencia de herramientas tradicionales como Ant o Maven, Gradle utiliza un sistema basado en scripts escritos en Groovy o Kotlin DSL, ofreciendo una mayor flexibilidad y un mejor rendimiento.

Durante el desarrollo de Food Store, Gradle fue utilizado para:

- Gestionar las dependencias del proyecto.
- Descargar automáticamente Hibernate, JPA y H2.
- Compilar el proyecto.
- Ejecutar la aplicación desde consola.
- Mantener una configuración centralizada del entorno de desarrollo.

El uso del Gradle Wrapper permite además ejecutar el proyecto sin necesidad de instalar Gradle previamente en el sistema, garantizando que todos los desarrolladores utilicen la misma versión de la herramienta.

3.4 JPA (Java Persistence API)

La Java Persistence API (JPA) es la especificación estándar de Java destinada al mapeo objeto-relacional (ORM). Su finalidad consiste en facilitar la persistencia de objetos Java dentro de una base de datos relacional sin necesidad de escribir consultas SQL para las operaciones básicas.

JPA define un conjunto de anotaciones que permiten establecer la correspondencia entre clases y tablas de la base de datos.

Entre las anotaciones utilizadas en este proyecto se destacan:

- `@Entity`
- `@Id`
- `@GeneratedValue`
- `@MappedSuperclass`
- `@OneToMany`
- `@ManyToOne`
- `@JoinColumn`

Gracias a estas anotaciones fue posible modelar las relaciones entre las distintas entidades del sistema de forma clara y mantenible.

En Food Store, JPA permitió abstraer la interacción con la base de datos, facilitando las operaciones CRUD sin necesidad de escribir código SQL para cada una de ellas.

3.5 Hibernate

Hibernate es una implementación de la especificación JPA que actúa como un framework ORM (Object Relational Mapping). Su función consiste en traducir automáticamente los objetos Java hacia registros de una base de datos relacional y viceversa.

El principal beneficio de Hibernate radica en eliminar la necesidad de manipular directamente consultas SQL para las operaciones más frecuentes, permitiendo que el desarrollador trabaje sobre objetos Java.

Entre las funcionalidades utilizadas en este proyecto se encuentran:

- Persistencia automática de entidades.
- Gestión de transacciones.
- Generación automática del esquema de la base de datos.

- Administración de relaciones entre entidades.
- Ejecución de consultas JPQL.

Hibernate permitió simplificar considerablemente el acceso a los datos y mejorar la mantenibilidad del código, ya que cualquier modificación realizada sobre una entidad puede reflejarse automáticamente en la base de datos mediante el contexto de persistencia.

3.6 H2 Database

H2 es un sistema gestor de bases de datos relacional liviano desarrollado en Java. Se caracteriza por ser una base de datos embebida que no requiere la instalación de un servidor independiente, lo que la convierte en una excelente opción para proyectos académicos y de desarrollo.

En Food Store se utilizó H2 en **modo archivo**, permitiendo que la información almacenada permanezca disponible entre distintas ejecuciones del programa.

Las principales ventajas de esta solución fueron:

- Instalación sencilla.
- Alta velocidad de ejecución.
- Integración directa con Hibernate.
- Creación automática de tablas mediante JPA.
- Persistencia permanente de la información.

Gracias a esta configuración fue posible almacenar categorías, productos, usuarios y pedidos sin depender de un motor de base de datos externo.

3.7 JPQL (Java Persistence Query Language)

JPQL es el lenguaje de consultas definido por JPA para realizar operaciones sobre entidades persistentes utilizando su modelo orientado a objetos.

A diferencia de SQL, JPQL trabaja directamente con clases y atributos del modelo de dominio, independientemente de la estructura física de la base de datos.

Durante el desarrollo del proyecto se implementaron diversas consultas utilizando JPQL, entre ellas:

- Búsqueda de productos por categoría.
- Búsqueda de usuarios por correo electrónico.
- Obtención de pedidos según su estado.
- Reportes de productos y pedidos.

La utilización de JPQL permitió desarrollar consultas reutilizables y desacopladas del motor de base de datos, facilitando el mantenimiento del sistema.

3.8 TypeScript

TypeScript es un lenguaje desarrollado por Microsoft que extiende JavaScript incorporando tipado estático y características avanzadas de programación orientada a objetos.

Una de sus principales ventajas consiste en detectar errores durante la etapa de compilación, reduciendo considerablemente los errores que podrían aparecer durante la ejecución.

En el frontend de Food Store se utilizaron interfaces TypeScript para representar entidades como:

- Producto
- Categoría
- Usuario
- Pedido
- Carrito

Esto permitió mantener un código más seguro, organizado y fácil de mantener.

3.9 Vite

Vite es una herramienta moderna para el desarrollo de aplicaciones web que proporciona un servidor de desarrollo extremadamente rápido y un sistema eficiente de empaquetado.

Entre sus principales características se destacan:

- Inicio casi instantáneo del servidor.
- Hot Module Replacement (HMR).
- Compilación optimizada.
- Configuración sencilla.

Durante el desarrollo de Food Store Vite fue utilizado como entorno de desarrollo del frontend, facilitando la creación de una aplicación modular organizada por páginas y componentes.

3.10 HTML5, CSS3 y LocalStorage

El frontend fue desarrollado utilizando HTML5 y CSS3 para la construcción de la interfaz gráfica del sistema.

HTML5 permitió estructurar las diferentes páginas de la aplicación, mientras que CSS3 fue utilizado para el diseño visual, logrando una interfaz organizada, intuitiva y agradable para el usuario.

Por otra parte, LocalStorage fue utilizado para almacenar información directamente en el navegador, permitiendo mantener:

- La sesión del usuario autenticado.
- El carrito de compras.
- Los pedidos simulados.
- Datos temporales necesarios para la navegación.

Esta solución permitió cumplir con los requerimientos de la consigna sin necesidad de conectar el frontend directamente con el backend, simulando el comportamiento de una aplicación persistente.

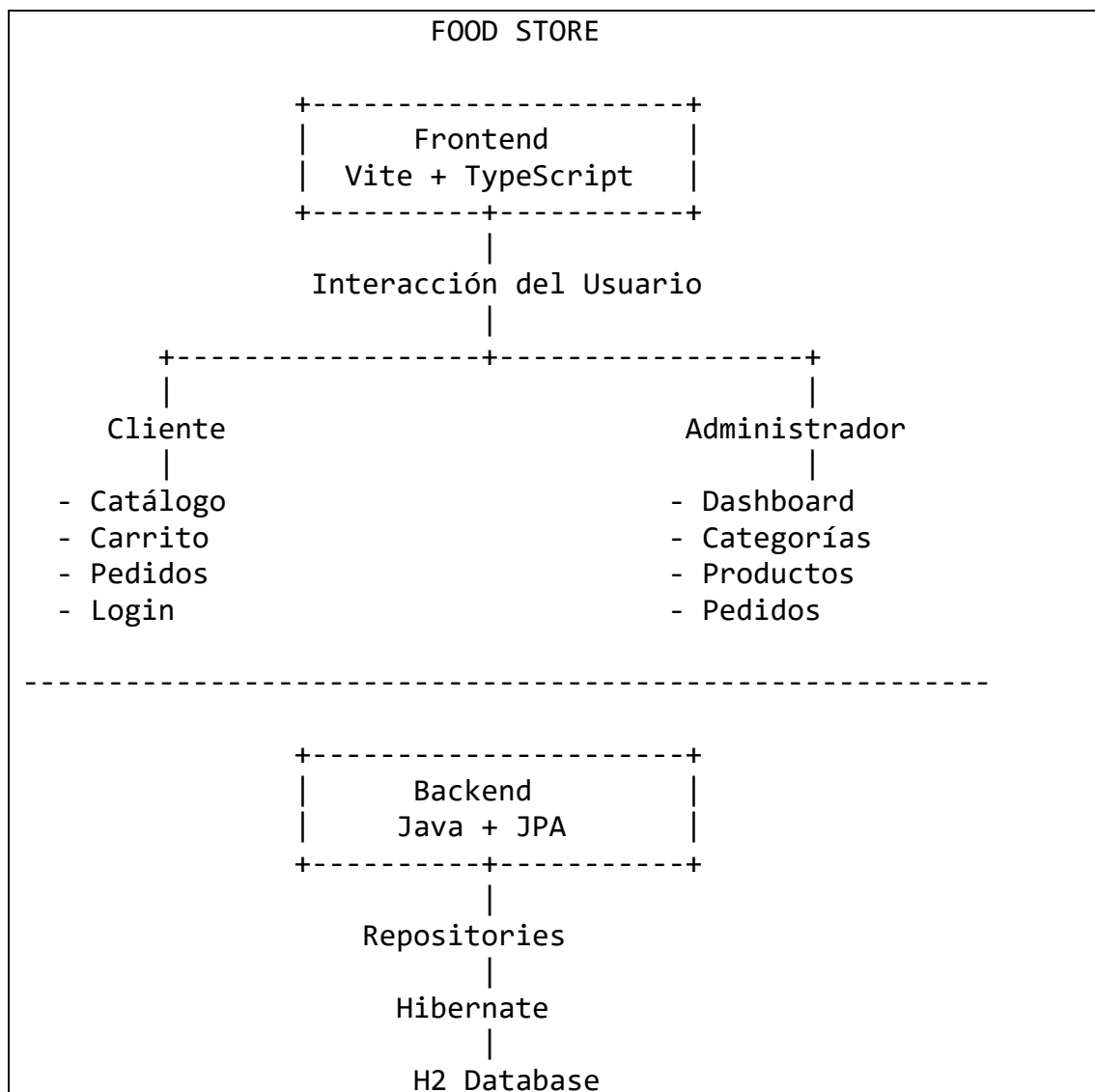
4. Arquitectura del Sistema y Decisiones Técnicas

4.1 Arquitectura General del Proyecto

Food Store fue desarrollado siguiendo una arquitectura dividida en dos aplicaciones independientes: un frontend responsable de la interacción con el usuario y un backend encargado de la lógica de negocio y la persistencia de datos.

Esta separación permite mantener desacopladas las responsabilidades de cada componente, facilitando tanto el mantenimiento como la escalabilidad del sistema.

La arquitectura implementada puede representarse de la siguiente manera:



La división entre frontend y backend permite que ambos proyectos evolucionen de forma independiente. En un escenario real, el frontend podría consumir los datos del backend mediante una API REST, manteniendo la misma estructura general del sistema.

4.2 Arquitectura del Backend

El backend fue desarrollado siguiendo una organización por capas, donde cada paquete posee una responsabilidad claramente definida.

La estructura implementada es la siguiente:

```
com.tp.jpjpa
|
├─ model
|
├─ model.enums
|
├─ repository
|
├─ util
|
└─ Maincom.tp.jpjpa
|
├─ model
|
├─ model.enums
|
├─ repository
|
├─ util
|
└─ Main
```

Cada componente cumple una función específica.

Model

Contiene todas las entidades del dominio del sistema.

Estas clases representan los objetos principales del negocio y son persistidas automáticamente mediante JPA.

Las entidades implementadas fueron:

- Base
- Categoria
- Producto
- Usuario
- Pedido
- DetallePedido

Enums

Se utilizaron enumeraciones para representar conjuntos de valores constantes dentro del sistema.

Entre ellas:

- Rol
- Estado
- FormaPago

Esto permitió evitar el uso de cadenas de texto para representar estados del sistema, mejorando la seguridad del código y reduciendo errores.

Repository

Toda la lógica de acceso a datos se concentró en la capa Repository.

Esta decisión permitió desacoplar completamente la lógica de persistencia de la lógica de negocio.

Cada repositorio se especializa en una entidad determinada y encapsula las operaciones de acceso a la base de datos.

Util

La clase JPAUtil centraliza la creación y administración del EntityManagerFactory, garantizando que toda la aplicación utilice una única instancia durante su ejecución.

Este enfoque evita la creación innecesaria de conexiones y mejora el rendimiento del sistema.

Main

La clase Main representa el punto de entrada del sistema.

Desde ella se implementó el menú interactivo solicitado por la consigna, permitiendo acceder a todas las funcionalidades del backend mediante una interfaz de consola.

4.3 Implementación del BaseRepository

Una de las principales decisiones de diseño fue implementar un repositorio genérico denominado **BaseRepository**.

En lugar de repetir las operaciones CRUD en cada entidad, se creó una clase abstracta capaz de reutilizar toda la lógica común.

Este repositorio implementa operaciones como:

- Guardar
- Buscar por ID
- Listar registros activos
- Baja lógica

Cada repositorio específico hereda estas funcionalidades, evitando duplicación de código y facilitando futuras ampliaciones.

Entre las principales ventajas de esta decisión se destacan:

- Mayor reutilización.
- Menor mantenimiento.
- Código más limpio.
- Mejor organización.
- Aplicación del principio DRY (Don't Repeat Yourself).

4.4 Repositorios Especializados

Aunque el BaseRepository resuelve las operaciones comunes, algunas entidades requieren consultas particulares.

Por esta razón se implementaron repositorios especializados como:

- CategoriaRepository
- ProductoRepository
- UsuarioRepository
- PedidoRepository

Cada uno incorpora únicamente las consultas específicas necesarias para su funcionamiento.

Por ejemplo:

- búsqueda de productos por categoría;
- búsqueda de usuarios por correo electrónico;
- búsqueda de pedidos por estado.

Esta organización evita sobrecargar el repositorio base con lógica específica de cada entidad.

4.5 Relaciones entre Entidades

Uno de los aspectos más importantes del backend fue el modelado correcto de las relaciones del dominio utilizando anotaciones JPA.

Las principales relaciones implementadas fueron:

Categoría → Productos

Una categoría puede contener múltiples productos.

Se implementó mediante una relación **OneToMany**.

Usuario → Pedidos

Cada usuario puede realizar múltiples pedidos.

Esto permitió mantener el historial de compras asociado a cada cliente.

Pedido → DetallePedido

Cada pedido está compuesto por uno o varios detalles.

Se implementó como una composición, ya que un detalle no puede existir independientemente del pedido al que pertenece.

DetallePedido → Producto

Cada detalle referencia un único producto.

Sin embargo, un mismo producto puede aparecer en múltiples pedidos.

Esta relación fue implementada mediante **ManyToOne**.

4.6 Persistencia mediante Hibernate

El acceso a la base de datos fue delegado completamente a Hibernate.

De esta forma, el desarrollador trabaja exclusivamente con objetos Java mientras que Hibernate se encarga de:

- generar las tablas;
- administrar las claves primarias;
- sincronizar entidades;
- gestionar transacciones;
- ejecutar consultas JPQL.

Este enfoque permitió reducir considerablemente la cantidad de código necesario para implementar la persistencia.

4.7 Arquitectura del Frontend

El frontend fue desarrollado utilizando Vite y TypeScript siguiendo una estructura modular organizada por responsabilidades.

La organización principal del proyecto es la siguiente:

```
src
├── pages
├── services
├── utils
├── types
├── components
└── data
```

Cada carpeta cumple un propósito específico.

Pages

Contiene todas las páginas de la aplicación.

Se encuentran organizadas según el tipo de usuario.

- auth
- store
- client
- admin

Esta estructura facilita la navegación y evita mezclar funcionalidades de diferentes módulos.

Services

La lógica de acceso a datos se centralizó mediante servicios.

Estos servicios encapsulan las operaciones relacionadas con productos, categorías, pedidos y usuarios.

Gracias a ello, las páginas no interactúan directamente con los archivos JSON o el LocalStorage.

Types

Se utilizaron interfaces TypeScript para representar las entidades del sistema.

Esto permitió disponer de tipado estático durante todo el desarrollo, detectando errores en tiempo de compilación.

Utils

Las funciones reutilizables fueron agrupadas en utilidades independientes.

Entre ellas se encuentran operaciones relacionadas con el LocalStorage, autenticación y otras tareas comunes.

Esta decisión evitó duplicación de código y favoreció la reutilización.

4.8 Uso de LocalStorage

La consigna del proyecto establece que el frontend debe funcionar de manera independiente utilizando archivos JSON y almacenamiento local.

Por este motivo se decidió utilizar LocalStorage para almacenar:

- sesión del usuario;
- carrito de compras;
- pedidos simulados;
- datos necesarios para mantener la navegación.

Gracias a este enfoque fue posible simular el comportamiento de una aplicación conectada a un backend real sin necesidad de implementar servicios REST.

4.9 Organización Modular

Una de las decisiones más importantes fue mantener completamente separadas las distintas responsabilidades del sistema.

En lugar de concentrar toda la lógica en pocas clases, cada módulo fue desarrollado de forma independiente.

Esta organización permitió obtener un proyecto:

- más legible;
- más mantenible;
- más escalable;
- más fácil de depurar.

Además, la estructura modular facilita futuras ampliaciones, como la incorporación de nuevos módulos o la sustitución del almacenamiento local por una API REST sin modificar significativamente la interfaz de usuario.

4.10 Justificación de las Decisiones Técnicas

Durante el desarrollo del proyecto se tomaron diversas decisiones con el objetivo de mejorar la calidad del software y facilitar su mantenimiento.

Entre las más relevantes se destacan:

Decisión	Justificación
BaseRepository genérico	Evitar duplicación de código y reutilizar operaciones CRUD.

Decisión	Justificación
Hibernate	Simplificar la persistencia mediante ORM.
JPA	Estandarizar el acceso a los datos utilizando anotaciones.
H2	Disponer de una base de datos liviana y fácil de configurar para un entorno académico.
TypeScript	Reducir errores mediante tipado estático.
Vite	Mejorar la velocidad de desarrollo y compilación del frontend.
LocalStorage	Cumplir con la consigna simulando persistencia en el navegador.
Organización modular	Favorecer el mantenimiento, la reutilización y la escalabilidad del proyecto.

5. Desarrollo del Backend

5.1 Introducción

El backend de **Food Store** fue desarrollado utilizando Java junto con la especificación JPA y el framework Hibernate para la gestión de la persistencia. La aplicación se ejecuta mediante un menú interactivo por consola, desde el cual es posible acceder a todas las funcionalidades del sistema.

Para organizar el código se adoptó una arquitectura por capas, separando claramente el modelo de dominio, la lógica de acceso a datos y las clases utilitarias. Esta organización permitió mantener un código modular, reutilizable y fácil de mantener.

El backend implementa la totalidad de las funcionalidades solicitadas por la consigna para la gestión de categorías, productos, usuarios y pedidos, incorporando además validaciones, consultas personalizadas y persistencia mediante una base de datos H2 en modo archivo.

5.2 Modelo de Dominio

El sistema fue modelado a partir de cinco entidades principales, representando los elementos fundamentales del negocio.

Las entidades implementadas son:

- Categoría
- Producto
- Usuario
- Pedido
- DetallePedido

Todas ellas heredan de una clase abstracta denominada **Base**, encargada de centralizar los atributos comunes del sistema.

Esta decisión permitió reutilizar código y mantener una estructura homogénea para todas las entidades persistentes.

La clase Base incorpora los siguientes atributos:

- Identificador único.
- Estado de eliminación lógica.
- Fecha de creación.

De esta manera, cualquier modificación relacionada con estos campos puede realizarse en un único lugar sin afectar el resto del sistema.

5.3 Gestión de Categorías

El módulo de categorías constituye el punto de partida para la organización del catálogo de productos.

Desde el menú principal el administrador puede realizar las siguientes operaciones:

- Alta de categorías.
- Consulta de categorías existentes.
- Modificación de categorías.
- Eliminación lógica.

La eliminación lógica fue implementada mediante un atributo booleano denominado **eliminado**, evitando la pérdida permanente de información y permitiendo conservar la integridad de las relaciones con otras entidades.

Para la persistencia se reutilizaron las operaciones genéricas implementadas en el BaseRepository, reduciendo considerablemente la cantidad de código repetido.

5.4 Gestión de Productos

Los productos representan los artículos disponibles dentro del catálogo.

Cada producto mantiene una relación directa con una categoría y almacena información como:

- Nombre.
- Descripción.
- Precio.
- Stock.
- Imagen.
- Estado de disponibilidad.

Durante la implementación se incorporaron diversas validaciones con el objetivo de preservar la consistencia de los datos.

Entre ellas se destacan:

- No permitir productos sin categoría.
- Validar el stock disponible.
- Evitar precios inválidos.
- Controlar la disponibilidad del producto.

Además del CRUD tradicional, se desarrolló una consulta JPQL que permite recuperar todos los productos pertenecientes a una categoría determinada, facilitando la generación de reportes y consultas específicas.

5.5 Gestión de Usuarios

El sistema incorpora una entidad Usuario encargada de representar tanto a los clientes como a los administradores.

Cada usuario almacena información personal como:

- Nombre.
- Apellido.
- Correo electrónico.
- Número telefónico.
- Contraseña.
- Rol.

Una de las principales funcionalidades implementadas consiste en la búsqueda de usuarios mediante su dirección de correo electrónico utilizando una consulta JPQL.

La utilización del correo electrónico como identificador permite evitar registros duplicados y simplifica futuras tareas de autenticación.

5.6 Gestión de Pedidos

El módulo de pedidos constituye el núcleo funcional del sistema.

Cada pedido está compuesto por uno o varios detalles, representando los productos seleccionados por el usuario durante una compra.

El pedido almacena información relacionada con:

- Fecha.
- Estado.
- Forma de pago.
- Total de la compra.

Cada detalle registra:

- Producto.
- Cantidad.
- Subtotal.

El cálculo del importe total se realiza automáticamente mediante el método `calcularTotal()`, definido por la interfaz `Calculable` e implementado por la entidad `Pedido`.

Este enfoque garantiza que el valor total permanezca sincronizado con los productos incluidos dentro del pedido.

5.7 Persistencia de Datos

Todas las operaciones de persistencia fueron implementadas utilizando JPA e Hibernate.

Cada repositorio administra su propio `EntityManager`, iniciando y finalizando las transacciones necesarias para cada operación.

Entre las operaciones implementadas se encuentran:

- Persistencia de nuevas entidades.
- Actualización de registros.
- Consultas por identificador.

- Listado de registros activos.
- Eliminación lógica.

Gracias al uso de Hibernate, la sincronización entre objetos Java y la base de datos se realiza de manera transparente para el desarrollador.

5.8 Consultas JPQL

Además de las operaciones CRUD tradicionales, el sistema implementa consultas específicas mediante Java Persistence Query Language (JPQL).

Entre las principales consultas desarrolladas se encuentran:

- Obtención de productos pertenecientes a una categoría determinada.
- Búsqueda de usuarios por dirección de correo electrónico.
- Obtención de pedidos según su estado.
- Reportes de pedidos por usuario.
- Reportes de productos organizados por categoría.

El uso de JPQL permitió trabajar directamente sobre las entidades del modelo de dominio, evitando depender de la estructura física de la base de datos y favoreciendo la portabilidad de la aplicación.

5.9 Validaciones Implementadas

Con el objetivo de garantizar la consistencia de la información almacenada, se incorporaron distintas validaciones durante el desarrollo del backend.

Entre las principales se destacan:

- Verificación de existencia antes de modificar registros.
- Validación de identificadores ingresados por el usuario.
- Control de productos activos.
- Validación de stock disponible antes de generar pedidos.
- Cálculo automático de subtotales y totales.
- Control de estados válidos para los pedidos.

Estas validaciones reducen significativamente la posibilidad de inconsistencias dentro de la base de datos y mejoran la experiencia de uso del sistema.

5.10 Menú Principal

La interacción con el backend se realiza mediante un menú de consola organizado en módulos.

Cada opción conduce a un submenú específico donde el usuario puede ejecutar las operaciones correspondientes.

La organización jerárquica del menú facilita la navegación y mantiene separadas las funcionalidades del sistema.

Los módulos principales son:

- Gestión de Categorías.
- Gestión de Productos.
- Gestión de Usuarios.
- Gestión de Pedidos.
- Reportes.
- Salida del sistema.

Este diseño permitió implementar una interfaz sencilla e intuitiva, adecuada para los requerimientos académicos del proyecto.

5.11 Resultados Obtenidos

Como resultado del desarrollo del backend se obtuvo un sistema completamente funcional capaz de administrar la información del negocio mediante persistencia en base de datos.

Las principales funcionalidades implementadas fueron:

- Gestión completa de categorías.
- Gestión completa de productos.
- Gestión completa de usuarios.
- Gestión completa de pedidos.
- Persistencia utilizando Hibernate.
- Consultas JPQL.
- Eliminación lógica.
- Organización modular mediante repositorios.
- Reutilización de código mediante BaseRepository.

La implementación de estas funcionalidades permitió cumplir con los objetivos planteados por la consigna y aplicar de forma práctica los conceptos desarrollados durante la asignatura.

```
===== MENU PRINCIPAL =====  
1. Categorías  
2. Productos  
3. Usuarios  
4. Pedidos  
5. Reportes  
0. Salir  
Opcion:
```

Figura 1. Menú principal del backend en ejecución.

```
----- Submenú CATEGORIAS -----  
1. Alta  
2. Baja Logica  
3. Modificacion  
4. Listado  
0. Volver  
Opcion: |
```

Figura 2. Menú secundario categorías

6. Desarrollo del Frontend

6.1 Introducción

El frontend de **Food Store** fue desarrollado utilizando **TypeScript**, **HTML5**, **CSS3** y **Vite**, con el objetivo de proporcionar una interfaz intuitiva y organizada para la interacción de los usuarios con el sistema.

La aplicación fue diseñada siguiendo una estructura modular, separando las distintas funcionalidades en páginas, servicios, tipos e utilidades. Esta organización permitió mantener el código desacoplado, facilitar su mantenimiento y favorecer la reutilización de componentes durante el desarrollo.

A diferencia del backend, el frontend utiliza archivos JSON y LocalStorage para simular la persistencia de datos y la autenticación, de acuerdo con los requerimientos establecidos por la consigna.

6.2 Organización del Proyecto

Con el objetivo de mantener una estructura clara y escalable, el código fuente fue organizado en carpetas según la responsabilidad de cada módulo.

La estructura principal del proyecto es la siguiente:

```
src/  
├── pages/  
├── services/  
├── types/  
├── utils/  
├── components/  
└── data/
```

Cada carpeta cumple una función específica dentro de la aplicación.

- **pages:** contiene las distintas vistas del sistema.
- **services:** centraliza el acceso a los datos.
- **types:** define las interfaces TypeScript utilizadas durante el desarrollo.
- **utils:** agrupa funciones reutilizables.
- **components:** contiene elementos compartidos entre distintas páginas.
- **data:** almacena los archivos JSON utilizados para simular la persistencia.

Esta organización permitió separar correctamente la lógica de presentación de la lógica de negocio, facilitando futuras ampliaciones del sistema.

6.3 Sistema de Autenticación

El sistema incorpora un mecanismo básico de autenticación implementado mediante LocalStorage.

Durante el inicio de sesión, el usuario ingresa su correo electrónico y contraseña. Estas credenciales son verificadas contra la información almacenada en el archivo de usuarios.

Una vez autenticado correctamente, la información de la sesión se almacena en LocalStorage, permitiendo mantener el acceso durante la navegación por la aplicación.

Dependiendo del rol del usuario autenticado, el sistema redirige automáticamente hacia la sección correspondiente.

Los roles implementados son:

- Administrador
- Usuario

Además, cada página protegida verifica la existencia de una sesión válida antes de permitir el acceso, evitando que usuarios no autenticados ingresen directamente mediante la URL.

6.4 Catálogo de Productos

El catálogo constituye la pantalla principal para los usuarios del sistema.

Desde esta sección es posible visualizar todos los productos disponibles organizados mediante un diseño en tarjetas.

Cada producto muestra información relevante como:

- Imagen.
- Nombre.
- Descripción.
- Precio.
- Estado de disponibilidad.

El catálogo incorpora distintas funcionalidades destinadas a mejorar la experiencia del usuario.

Entre ellas se destacan:

- búsqueda dinámica;
- filtrado por categorías;
- ordenamiento alfabético;
- ordenamiento por precio;
- contador de productos encontrados.

Estas herramientas permiten localizar rápidamente los productos deseados incluso cuando el catálogo contiene una gran cantidad de elementos.

6.5 Detalle del Producto

Cada producto posee una pantalla de detalle donde se presenta información ampliada respecto al artículo seleccionado.

En esta vista se muestran:

- imagen en mayor tamaño;
- descripción completa;
- precio;
- stock disponible;
- estado del producto.

Además, el usuario puede seleccionar la cantidad que desea agregar al carrito.

Antes de confirmar la operación, el sistema valida que:

- exista stock suficiente;
- el producto se encuentre disponible;
- la cantidad seleccionada sea válida.

Estas verificaciones evitan inconsistencias durante el proceso de compra.

6.6 Carrito de Compras

El carrito representa uno de los módulos principales del sistema.

Cada vez que un usuario incorpora un producto, este queda almacenado en LocalStorage para mantener la información incluso después de cerrar el navegador.

Desde esta pantalla el usuario puede:

- modificar cantidades;
- eliminar productos;
- vaciar completamente el carrito;
- visualizar el total de la compra.

Cada modificación realizada actualiza automáticamente el importe total del pedido, brindando información inmediata al usuario.

En caso de no existir productos cargados, la aplicación muestra un mensaje indicando que el carrito se encuentra vacío junto con un acceso directo para regresar al catálogo.

6.7 Historial de Pedidos

Una vez confirmada una compra, el sistema registra el pedido correspondiente permitiendo al usuario consultar posteriormente su historial.

Cada pedido informa:

- número de pedido;
- fecha;
- estado;
- productos incluidos;
- importe total.

Los estados implementados permiten representar las distintas etapas del proceso de compra:

- Pendiente.
- En preparación.
- Entregado.
- Cancelado.

Esta funcionalidad mejora la experiencia del usuario al permitir realizar un seguimiento del estado de sus pedidos.

6.8 Panel de Administración

El sistema incorpora un panel exclusivo para usuarios con rol de administrador.

Desde esta sección es posible gestionar toda la información del sistema mediante diferentes módulos administrativos.

El Dashboard presenta un resumen general del funcionamiento de la aplicación mostrando estadísticas relacionadas con:

- cantidad de categorías;
- cantidad de productos;
- pedidos registrados;
- productos disponibles.

Este resumen permite obtener una visión rápida del estado general del sistema.

6.9 Gestión de Categorías

El módulo de categorías permite realizar las operaciones básicas de administración.

Las funcionalidades implementadas incluyen:

- alta;
- modificación;
- eliminación;
- visualización de categorías existentes.

La información se presenta mediante una tabla donde se muestran los principales atributos de cada categoría junto con las acciones disponibles.

6.10 Gestión de Productos

El módulo de productos permite administrar completamente el catálogo.

Cada registro muestra información relacionada con:

- imagen;
- nombre;
- categoría;
- precio;
- stock;
- disponibilidad.

Desde esta pantalla el administrador puede realizar todas las operaciones CRUD requeridas por la consigna.

6.11 Gestión de Pedidos

El administrador dispone de una vista específica para consultar todos los pedidos registrados por los distintos usuarios.

La información puede filtrarse según el estado del pedido, facilitando la administración de las compras realizadas.

Cada pedido presenta:

- cliente;
- fecha;
- estado;

- cantidad de productos;
- importe total.

Esta funcionalidad permite realizar el seguimiento del proceso completo de venta.

6.12 Uso de LocalStorage

Una de las decisiones adoptadas durante el desarrollo fue utilizar LocalStorage como mecanismo de persistencia para el frontend.

Esta solución permitió almacenar:

- sesión del usuario;
- carrito de compras;
- pedidos simulados;
- información temporal utilizada por la aplicación.

El uso de LocalStorage permitió cumplir con los requerimientos académicos del proyecto sin necesidad de conectar el frontend directamente con el backend mediante servicios web.

6.13 Experiencia de Usuario

Durante el desarrollo se buscó construir una interfaz clara e intuitiva.

Para ello se procuró mantener una navegación consistente entre todas las pantallas, utilizando una distribución uniforme de botones, formularios, tablas y tarjetas.

Asimismo, las validaciones implementadas proporcionan retroalimentación inmediata al usuario ante errores de ingreso de datos o acciones no permitidas, mejorando significativamente la usabilidad del sistema.

6.14 Resultados Obtenidos

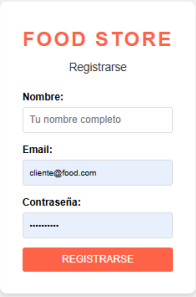
El desarrollo del frontend permitió implementar la totalidad de las funcionalidades previstas para los distintos perfiles del sistema.

Entre ellas se destacan:

- autenticación por roles;
- navegación del catálogo;
- búsqueda y filtros;
- carrito persistente;
- historial de pedidos;
- dashboard administrativo;
- gestión de categorías;
- gestión de productos;
- gestión de pedidos.

Gracias a la organización modular y al uso de TypeScript fue posible desarrollar una aplicación mantenible, escalable y alineada con los objetivos planteados por el Trabajo Práctico Integrador.

Registro



The image shows a registration form for 'FOOD STORE'. The form is centered on a light gray background. It has a white background with a subtle shadow. The title 'FOOD STORE' is in red, and 'Registrarse' is in a smaller, gray font below it. The form contains three input fields: 'Nombre:' with a placeholder 'Tu nombre completo', 'Email:' with a placeholder 'cliente@food.com', and 'Contraseña:' with a placeholder '.....'. Each input field has a light blue border. At the bottom of the form is a red button with the text 'REGISTRARSE' in white capital letters.

Iniciar Sesión

FOOD STORE

Iniciar Sesión

Email:

tucorreo@ejemplo.com

Contraseña:

Mínimo 8 caracteres

INGRESAR

¿No tenés cuenta? [Registrate acá](#)

Home

FOOD STORE

Mis PedidosCarritoCerrar sesión

Categorías

Todos

Hamburguesas

Pizzas

Empanadas

Papas Fritas

Lomos

Buscar productos...

Ordenar

Nuestros Productos

Elegí tus favoritos y agregalos al carrito.

Hamburguesa Clásica

Hamburguesas

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$9.999

-

0

+

Hamburguesa Doble

Hamburguesas

Doble carne 150g c/u, cheddar, panceta y salsa barbaeo

\$6.200

-

0

+

Hamburguesa Completa

Hamburguesas

Carne 150g, huevo, jamón, queso, lechuga y tomate

\$5.500

-

0

+

Hamburguesa Veggie

Hamburguesas

Medallón de lentejas y quinoa, rúcula, tomate confitado y hummus

\$5.000

-

0

+

Pedidos

FOOD STORE

Mis PedidosCarrito

Historial de Pedidos

Consultá el estado de todas tus compras.

Pedido #1782925826086

PENDIENTE

Fecha

1/7/2026

Productos

2

Total

\$10.700

Productos del pedido

Hamburguesa Completa x1

\$5.500

Pizza Especial x1

\$5.200

Pedido #1782836791176

CANCELADO

Fecha

30/6/2026

Productos

1

Total

\$3.800

Productos del pedido

Pizza Mozzarella x1

\$3.800

34

Carrito

FOOD STORE

Mis PedidosCarritoCatálogoCerrar sesión

Tu Carrito

Revisá tus productos antes de finalizar la compra.

Hamburguesa Clásica

Precio unitario

\$9.999

-

1

+

Subtotal

\$9.999

Hamburguesa Doble

Precio unitario

\$6.200

-

1

+

Subtotal

\$6.200

Resumen

Subtotal: \$21.699

\$21.699

Finalizar compra

Vaciar carrito

Panel Admin

FOOD STORE

Administración

Dashboard

Categorías

Productos

Pedidos

Admin

Ir a Tienda

Cerrar sesión

Dashboard

Panel de Administración

Gestión centralizada de Food Store.

Categorías

5

Gestionar

Productos

20

Gestionar

Pedidos

8

Gestionar

Disponibles

19

Ver Productos

Resumen General

Categorías activas

5

Productos activos

1

Productos inactivos

19

Pedidos por Estado

Pendientes

3

Confirmados

1

Terminados

9

Cancelados

1

Panel Admin-Productos

FOOD STORE

Administración

Dashboard

Categorías

Productos

Pedidos

Admin

Tienda

Salir

Productos

Administración de productos

Buscar producto...

Todas las categorías

Todos

Nuevo Producto

ID	Imagen	Nombre	Descripción	Precio	Categoría	Stock	Estado	Acciones
1		Hamburguesa Clásica	Carne 150g, lechuga, tomate, cebolla morada y mayonesa	\$9.999	Hamburguesas	50	Disponible	
2		Hamburguesa Doble	Doble carne 150g c/u, cheddar, panceta y salsa barbacoa	\$6.200	Hamburguesas	40	Disponible	
3		Hamburguesa Completa	Carne 150g, huevo, jamón, queso, lechuga y tomate	\$5.500	Hamburguesas	30	Disponible	
4		Hamburguesa Veggie	Medallón de lentejas y quinoa, rúcula, tomate confitado y hummus	\$5.000	Hamburguesas	20	Disponible	
5		Pizza Mozzarella	Mozzarella, salsa de tomate y orégano	\$3.800	Pizzas	25	Disponible	
6		Pizza Napolitana	Mozzarella, rodajas de tomate, ajo, aceitunas y albahaca	\$4.500	Pizzas	25	Disponible	

35

Panel Admin-Pedidos

FOOD STORE

Administración

Dashboard

Categorías

Productos

Pedidos

Gestión de Pedidos

Administración de pedidos

AdminTiendaSalir

Buscar cliente...Todos los estados

Pedido #1782925826086

Pendiente

2\$10,700

Ver detalleCambiar estado

Pedido #1782836791176

Cancelado

1\$3,800

Ver detalleCambiar estado

Pedido #1782836587822

Confirmado

5\$30,500

Panel Admin-Categorías

FOOD STORE

Administración

Dashboard

Categorías

Productos

Pedidos

Categorías

Administración del catálogo

AdminTiendaSalir

Buscar categoría...Toda+Nueva Categoría

ID	Nombre	Descripción	Estado	Acciones
1	Hamburguesas	Hamburguesas artesanales con carne premium	Activa	
2	Pizzas	Pizzas al horno de barro	Activa	
3	Empanadas	Empanadas caseras homeadas	Activa	
5	Papas Fritas	Acompañamientos crujientes	Activa	
6	Lomos	Sandwichs de carne	Activa	

7. Dificultades Encontradas y Soluciones Implementadas

7.1 Introducción

Durante el desarrollo de Food Store surgieron distintos desafíos técnicos relacionados tanto con el backend como con el frontend. La mayoría de ellos estuvieron asociados a la configuración del entorno de desarrollo, la persistencia de datos, el modelado de entidades y la organización del código.

Cada inconveniente representó una oportunidad para profundizar en el funcionamiento de las herramientas utilizadas y mejorar la calidad general del proyecto. A continuación se describen las principales dificultades encontradas y las soluciones adoptadas para resolverlas.

7.2 Configuración Inicial del Proyecto

Uno de los primeros desafíos consistió en configurar correctamente el entorno de desarrollo para que todas las dependencias del proyecto funcionaran de manera integrada.

En el backend fue necesario configurar Gradle junto con Hibernate, JPA y la base de datos H2, asegurando que todas las bibliotecas fueran compatibles entre sí.

En el frontend también fue necesario configurar adecuadamente Vite, TypeScript y la estructura inicial del proyecto para permitir una organización modular desde el comienzo.

Una vez resueltos estos aspectos, fue posible avanzar con el desarrollo de las funcionalidades sin inconvenientes relacionados con el entorno de ejecución.

7.3 Modelado de las Relaciones entre Entidades

Una de las principales dificultades técnicas estuvo relacionada con el modelado de las relaciones entre las entidades del sistema.

Durante las primeras pruebas fue necesario analizar cuidadosamente la cardinalidad correspondiente a cada relación para garantizar un comportamiento correcto de la persistencia.

Especial atención requirió la implementación de las asociaciones entre:

- Categoría y Producto.
- Usuario y Pedido.
- Pedido y DetallePedido.

La correcta utilización de las anotaciones de JPA permitió representar adecuadamente dichas relaciones y evitar inconsistencias en la base de datos.

Asimismo, fue necesario comprender el comportamiento de las operaciones en cascada y el ciclo de vida de las entidades administradas por Hibernate.

7.4 Implementación del BaseRepository

Otro desafío importante consistió en desarrollar un repositorio genérico reutilizable.

Inicialmente cada entidad podía haber implementado sus propias operaciones CRUD, pero esto habría generado una considerable duplicación de código.

Como solución se diseñó un BaseRepository capaz de centralizar todas las operaciones comunes.

Esta decisión permitió:

- reducir la repetición de código;
- facilitar futuras modificaciones;
- mantener una estructura homogénea entre todos los repositorios.

La reutilización obtenida mediante esta implementación mejoró significativamente la mantenibilidad del proyecto.

7.5 Consultas JPQL

La implementación de consultas personalizadas mediante JPQL también representó un desafío durante el desarrollo.

Fue necesario comprender la diferencia entre SQL tradicional y JPQL, ya que este último trabaja directamente sobre las entidades del modelo de dominio en lugar de hacerlo sobre las tablas físicas de la base de datos.

Luego de realizar diversas pruebas fue posible implementar correctamente consultas como:

- búsqueda de productos por categoría;
- búsqueda de usuarios por correo electrónico;
- obtención de pedidos por estado.

Estas consultas permitieron ampliar las funcionalidades del sistema manteniendo un bajo nivel de acoplamiento con la base de datos.

7.6 Persistencia mediante H2

Durante las primeras etapas del proyecto también surgieron dificultades relacionadas con la persistencia de la información.

La configuración de Hibernate y H2 debía garantizar que los datos permanecieran almacenados entre distintas ejecuciones de la aplicación.

Luego de revisar la configuración del archivo de persistencia fue posible utilizar correctamente una base de datos H2 en modo archivo, logrando que las entidades persistieran de forma permanente sin requerir un servidor de base de datos externo.

Esta solución simplificó considerablemente el proceso de desarrollo y permitió cumplir con los requerimientos académicos establecidos por la cátedra.

7.7 Organización del Frontend

En el frontend uno de los principales desafíos fue mantener una estructura organizada a medida que el proyecto aumentaba de tamaño.

Inicialmente muchas funcionalidades tendían a concentrarse en pocas páginas, dificultando el mantenimiento del código.

Para solucionar este inconveniente se reorganizó la aplicación utilizando una arquitectura modular basada en:

- páginas;
- servicios;
- interfaces;
- utilidades;
- componentes reutilizables.

Esta reorganización permitió separar claramente las responsabilidades de cada módulo y simplificó la incorporación de nuevas funcionalidades.

7.8 Manejo del LocalStorage

La utilización de LocalStorage para simular la persistencia del frontend también presentó algunos desafíos.

Fue necesario diseñar una estrategia que permitiera mantener sincronizados:

- el usuario autenticado;
- el carrito de compras;
- los pedidos simulados.

Además, se implementaron funciones auxiliares para centralizar el acceso al almacenamiento local, evitando duplicación de código y facilitando futuras modificaciones.

Gracias a esta organización fue posible mantener la información disponible incluso después de cerrar el navegador.

7.9 Gestión del Carrito de Compras

La implementación del carrito requirió controlar diversas situaciones particulares.

Entre ellas:

- actualización automática de cantidades;
- eliminación de productos;
- cálculo dinámico del total;
- persistencia entre sesiones.

También fue necesario validar la disponibilidad del stock antes de permitir incorporar nuevos productos al carrito, evitando inconsistencias durante el proceso de compra.

7.10 Validaciones del Sistema

Otro aspecto importante fue la incorporación de validaciones tanto en el frontend como en el backend.

Estas validaciones permitieron prevenir errores frecuentes relacionados con:

- campos obligatorios;
- datos inválidos;
- cantidades negativas;
- stock insuficiente;
- autenticación;
- permisos de acceso.

La implementación de estas verificaciones mejoró considerablemente la robustez y confiabilidad del sistema.

7.11 Aprendizajes Obtenidos

El desarrollo del proyecto permitió integrar conocimientos adquiridos durante toda la carrera y comprender con mayor profundidad el funcionamiento de las tecnologías utilizadas.

Entre los principales aprendizajes pueden destacarse:

- modelado de entidades mediante JPA;
- utilización de Hibernate como framework ORM;
- implementación de consultas JPQL;
- organización modular de aplicaciones;
- separación entre frontend y backend;
- reutilización mediante clases genéricas;
- aplicación de principios de programación orientada a objetos;
- gestión de persistencia utilizando H2;
- desarrollo de interfaces web utilizando TypeScript y Vite.

Asimismo, el proyecto permitió fortalecer habilidades relacionadas con la resolución de problemas, depuración de código, análisis de errores y mejora continua del software.

7.12 Reflexión Final

Cada uno de los desafíos encontrados durante el desarrollo contribuyó al crecimiento técnico obtenido a lo largo del proyecto.

Lejos de representar un obstáculo, los inconvenientes surgidos permitieron comprender con mayor profundidad el funcionamiento interno de las tecnologías utilizadas y reforzar conceptos fundamentales de programación orientada a objetos, persistencia de datos y desarrollo de aplicaciones Full Stack.

Como resultado se obtuvo una aplicación funcional, organizada y alineada con los objetivos propuestos por el Trabajo Práctico Integrador, constituyendo una experiencia significativa tanto desde el punto de vista académico como profesional.

8. Conclusiones

Conclusión

El desarrollo del proyecto **Food Store** representó una experiencia integradora que permitió aplicar de manera práctica los conocimientos adquiridos a lo largo de la asignatura Programación III y de otras materias de la carrera.

Durante su implementación fue posible combinar conceptos de programación orientada a objetos, persistencia de datos, desarrollo web y organización modular del software, integrando tecnologías tanto para el frontend como para el backend en una única solución.

En el backend se consolidaron conocimientos relacionados con el uso de Java, JPA e Hibernate para el modelado de entidades, la implementación de relaciones entre objetos, la reutilización de código mediante repositorios genéricos y la utilización de consultas JPQL para resolver requerimientos específicos del sistema.

Por otra parte, el desarrollo del frontend permitió profundizar en el uso de TypeScript, Vite y LocalStorage para construir una interfaz organizada, dinámica y orientada a la experiencia del usuario. La implementación de funcionalidades como autenticación, catálogo de productos, carrito de compras y panel de administración permitió comprender la importancia de mantener una correcta separación entre la lógica de negocio y la presentación de la información.

Además de los aspectos técnicos, el proyecto contribuyó al fortalecimiento de habilidades relacionadas con el análisis de problemas, la depuración de errores, la organización del código y la toma de decisiones durante el desarrollo de software.

Uno de los aprendizajes más importantes fue comprender la relevancia de diseñar una arquitectura modular desde el inicio del proyecto. Esta decisión facilitó el mantenimiento del sistema, permitió reutilizar componentes y simplificó la incorporación de nuevas funcionalidades a medida que avanzaba el desarrollo.

Asimismo, la experiencia permitió valorar la importancia de la documentación técnica como parte fundamental del ciclo de vida del software, ya que constituye una herramienta esencial para comprender el funcionamiento interno del sistema y facilitar futuras tareas de mantenimiento.

Finalmente, puede concluirse que se lograron cumplir los objetivos planteados por el Trabajo Práctico Integrador, obteniendo una aplicación funcional que implementa las principales características solicitadas por la consigna y que refleja la aplicación práctica de los contenidos desarrollados durante la carrera.

Link Video :

<https://drive.google.com/file/d/1u2KjZeWNfZr2lJxC875uzOOPu8T03eH-/view?usp=sharing>

Link Repositorio GitHub:

<https://github.com/ChrisOlivero/Ecommerce-FoodStore>

9. Bibliografía y Webgrafía

Para el desarrollo del proyecto se consultó documentación oficial y recursos técnicos relacionados con las tecnologías utilizadas.

Bibliografía

- Oracle. (2025). *Java Documentation*. <https://docs.oracle.com/en/java/>
 - Eclipse Foundation. (2025). *Jakarta Persistence Specification*. <https://jakarta.ee/specifications/persistence/>
 - Hibernate. (2025). *Hibernate ORM Documentation*. <https://hibernate.org/orm/documentation/>
 - H2 Database. (2025). *H2 Database Engine*. <https://www.h2database.com/>
 - Gradle Inc. (2025). *Gradle User Manual*. <https://docs.gradle.org/>
 - Microsoft. (2025). *TypeScript Handbook*. <https://www.typescriptlang.org/docs/>
 - Vite Team. (2025). *Vite Documentation*. <https://vite.dev/guide/>
 - MDN Web Docs. (2025). *HTML, CSS y JavaScript*. <https://developer.mozilla.org/>
 - OpenJDK. (2025). *Java Platform Documentation*. <https://openjdk.org/>
 - Git SCM. (2025). *Git Documentation*. <https://git-scm.com/doc>
-

Recursos utilizados

Durante el desarrollo del proyecto también se utilizaron las siguientes herramientas de apoyo:

- Visual Studio Code.
- IntelliJ IDEA / NetBeans (según el entorno utilizado para el backend).
- Git y GitHub para el control de versiones.
- H2 Database.
- Gradle Wrapper.
- Navegadores basados en Chromium para pruebas del frontend.